

A Parallel GPU Algorithm for the Stepping Stone Puzzle

Peter Weiblen

University of Minnesota

+1 651-528-4703

weibl021@umn.edu

Adam Meyers

University of Minnesota

+1 651-343-7545

meye3224@umn.edu

ABSTRACT

The stepping stone puzzle features a state space that rapidly grows as the input increases. The only known way to solve this problem is to check all possible solution states for a given input N , with $N \geq 7$ not yet being solved. We propose a parallel GPU algorithm for solving the stepping stone puzzle, with each GPU thread taking in a board and checking the entire state space of said board. This proves to be more efficient than an equivalent CPU implementation, solving $N = 5$ in ~ 1.3 hours, compared to ~ 4 hours by the previous best algorithm. Extrapolations of computation time for $N \geq 6$ also result in theoretically saved time. However, it is still not enough for solving $N = 7$ in a reasonable timeframe. Further optimization is needed to solve this problem at $N \geq 7$.

1. INTRODUCTION

The stepping stone puzzle is a math problem set on an infinite grid. You are given an integer N as input, which defines the amount of ones that can be placed within cells on said grid. After placing all N ones, you must then place the integers 2, 3, 4, $\dots$ $m-1$, m on the grid in ascending order, following the restriction that the 8 neighboring tiles must add up to the newly placed number. The goal of the stepping stone puzzle is to find the maximum number m that can be placed with a given integer N .

This may seem like a fairly simple puzzle. However, the only known way to find the maximum number m for any given N is to check every single possible state that can be reached starting from N ones, or at least logically eliminate the states as possibilities. This state space grows at a high exponential rate with respect to N , which leads to incredibly long computation times to search all possible states. The best known algorithm, designed by Hugo van der Sanden [1], takes 10 weeks to search the state space of $N = 6$ and find the solution of $m = 60$. $N = 7$ has yet to

be solved, with an estimated computation time of about 44 years using the aforementioned algorithm.

We believe that this algorithm can be improved. After just a few levels of depth generating boards, there are already thousands of unique boards whose state space must be searched. Generating and checking additional boards does not require any interaction between the already generated boards, besides comparing final m values to find the board with the maximum possible m . Due to these facts, the stepping stone puzzle seems like a prime candidate for parallelism. In this paper, we propose a GPU-based parallel algorithm for the stepping stone puzzle. [5]

2. DESIGN OVERVIEW

First it should be covered how exactly board states are being enumerated. We elected to use the same general method that has been used before, which is to focus only on placing the next integer needed and insert the ones lazily as needed. By iterating over all positions for the next element and all valid permutations of the ones around it, we can exhaust all possible board states. As long as it is done carefully, this is, in addition to being equivalent to placing the ones at the beginning, a much more efficient way of enumerating through all possible boards, since no time is wasted on subtly different one positions which yield equivalent dead end states. The other reason for constructing the board state in this manner is that it makes it computationally simple to traverse both “down” the tree, adding elements, and “up” the tree removing elements without losing our place.

The general architecture is fairly straightforward, on the CPU side of things we start with an empty board, the root of the state space. We generate all possible boards breadth first, to some appropriate fixed depth. In this way we generate the states for the GPU to operate on. On the GPU side each

thread is given a single board whose state space is searched depth first.

3. IMPLEMENTATION

3.1 Boards

In order for each thread to properly search the state space of each board, a necessity is to have some sort of way to store all of the information contained in said board. The algorithm requires the ability to not just generate new boards from an existing board, but also to traverse backwards to find the boards that another board was created from. Thus, board information must be stored in such a way that the algorithm is capable of efficiently carrying out both of these functions.

One solution would be to store the entire utilized section of the grid as a matrix of integers. However, this method of storage often takes up a large amount of space storing useless information such as cells that have no assigned elements. Instead, the necessary information to store for each board was narrowed down to a few necessary values. Three integers were needed per board: the total number of ones available, the total number of ones already placed, and the highest number currently placed in the grid. In addition to these three integers per board, each element placed on the board needed to store its value, its x and y coordinates on the board, and a flag to determine if it was on a multiboard or not.

Originally, all four of these per-element values were stored in separate arrays of a size corresponding to the maximum expected number m that could be reached for the given N . This maximum value was typically set to around 90, as that is higher than the known solutions for $N \leq 6$, and notably higher than what the $N = 7$ solution is expected to be, providing plenty of wiggle room for potential maximum solutions. However, keeping 4 separate arrays took a lot of memory, and caused notable amounts of memory latency when running on the GPU. In order to fix this problem, bitwise operators were used to pack all of this information into a single 32-bit integer for each element, allowing all of the element information to be stored in a single array, and only adding a few bitwise operations to the kernel for the purpose of packing and unpacking values.

After packing, this meant each board could be stored as just three 32-bit integers and one 32-bit integer array of length 90. This reduced storage space enough to store all of the board information solely on the thread's registers, allowing for very quick memory access, significantly improving the performance of the kernel.

3.2 Deduplication

For the purposes of finding optimal boards, we need not consider duplicates, boards which are equivalent up to rotation or reflection. Fortunately, it is not too costly to ensure our set of boards has no such duplicates. For checking whether two boards are equivalent it suffices to, for each of the eight possible orientations, go through and see that each element is translated the same amount from one board to the other. In this way the check only requires constant memory and linear time with respect to the board's depth.

To check if our entire set of boards at a given depth has any duplicates seems like it might be quite computationally expensive. However it can be done in linear time with respect to the number of boards by making a key observation. If two boards are equivalent, removing one element does not change this fact. Thus, assuming the boards from the previous depth were already deduplicated, new duplicate boards can only be generated from the same board, and therefore we only need to check for duplicates between those boards which are generated from the same previous board.

3.3 Multiboards

For $N \geq 5$ the method of inserting ones lazily as needed presents a new complication. It is no longer sufficient to insert elements directly next to existing elements and add ones as needed. This is because after placing a two cell with ones around it it is now possible to place a three cell any arbitrary distance away and populate three of the remaining ones around it to satisfy the sum requirement. The solution to this in our program is what we refer to as a "multiboard".

When the possibility arises for placing an element an arbitrary distance away, we can store two boards instead of one. In practice we don't even need to actually store two boards. Since none of the elements between the boards overlap, we can

simply set a bit flag in the element's packed information integer that denotes which virtual board a particular element belongs to. This does not significantly complicate the board generation, however. Three new functions are needed. One function determines if a board can be split when adding a new element, and generates those new multiboards. Another function adds elements to multiboards. This one is not dissimilar to adding elements to regular boards, though some extra care must be taken when finding open positions. Finally the most taxing function is that which merges multiboards back into regular boards. For this a common position must be found on both boards, new one elements must be accounted for, all possible orientations must be checked, and finally we must ensure that there are no collisions when overlaying the two boards.

3.4 Search Kernel

To send the data between the CPU and the GPU an intermediate one dimensional array of integers is used. This array is interleaved to maximize the efficiency of the kernel code which moves the data into the registers.

Many of the kernel functions are modified versions of the CPU board generation code, rewritten to reduce warp divergence as much as possible. In an effort to keep the kernel code as simple as possible there is no deduplication and no handling of multiboards. Indeed it is not necessary to handle these things on the GPU since after a certain depth no new duplicates and no new multiboards can possibly be generated. It is thus sufficient to generate to this depth on the CPU, and continue searching the multiboards' state spaces until all have merged back to regular boards or have hit dead ends.

Another complication with the GPU code was keeping track of the maximum board. Initially it was attempted to have a global maximum board that would be updated atomically, but this proved cumbersome and was dropped in favor of using the input array as output as well. Since the boards are moved into the registers at the beginning, this global memory wasn't being used for anything else, and this way the only necessary atomic operation is updating a single integer which contains the depth of the maximal board found so far.

4. VERIFICATION

No automatic verification system was designed for use with this algorithm. Instead, the maximal board is simply printed to the console, allowing for manual verification that the board accurately follows the rules of the stepping stone puzzle, and that the maximum placed number matches the known solutions for $N \leq 6$, as well as the known bounds for $N \geq 7$ [3]. As this algorithm will only be realistically run on less than 10 values of N , such manual verification was deemed as acceptable, with no need for further automation.

For $N \leq 6$, solutions can easily be verified by comparing with the already proven solutions shown below.

Table 1. Known solutions for $N \leq 6$

N	1	2	3	4	5	6
m	1	16	28	38	49	60

For $N \geq 7$, there is no easy way to verify that a solution is the maximal solution, though there are still known lower bounds that can be used to discount many incorrect solutions. For example, it is known that the maximal value of m when $N = 7$ is at least $m = 71$, so any returned board with a maximum $m < 71$ can easily be disregarded. In order to prove that a solution for $N \geq 7$ is the maximal solution, we must ensure that the algorithm checks all possible solution states, and additionally contains no errors that could possibly result in suboptimal boards being returned as solutions. If these facts can be ensured, then the algorithm itself can be used as a proof for the solution.

5. PERFORMANCE

The CPU-only and GPU-assisted versions of our algorithm were run on a computer with an AMD Ryzen 7 7700 and NVIDIA GeForce RTX 4060 Ti. For $N = 2$ and $N = 3$, it was run 5 times, with the mean runtimes shown in the table below. For $N = 4$ (CPU and GPU-assisted) and $N = 5$ (GPU-assisted) it was only run once each. $N = 5$ (CPU) and $N \geq 6$ have not yet been completed, due to time constraints. Instead, extrapolated times based on estimations of the search space size are

shown below. These are compared with the times for Hugo van der Sanden’s algorithm. It should be noted that the times for Sanden’s algorithm were most likely run on different hardware.

Table 2. Performance of Hugo van der Sanden’s algorithm, and our algorithms (CPU-only and GPU-assisted) for $N \leq 7$.

N	Sanden’s Algorithm	CPU-Only Algorithm	GPU-Assisted Algorithm
2	<1 s	0.00164 s	0.115 s
3	<1 s	0.924 s	0.826 s
4	~42 s	383 s (6 min)	108 s (2 min)
5	~14000 s (4 hours)	$1.3 * 10^5$ s (1.5 days) [†]	4574 s (1.3 hours)
6	~ $6.0 * 10^6$ s (10 weeks)	$5.5 * 10^7$ s (1.7 years) [†]	$9.0 * 10^5$ s (10 days) [†]
7	~ $1.4 * 10^9$ s (44 years) [†]	$1.3 * 10^{10}$ s (412 years) [†]	$2.0 * 10^8$ s (6.3 years) [†]

[†] Extrapolated runtime based on estimations of search space.

At $N = 2$, the GPU-assisted program takes a longer time to run than the CPU-only version. This is likely due to the extra computation time spent copying boards to and from the GPU, which significantly outweighs the time spent searching due to the relatively small state space at $N = 2$. At $N \geq 3$ the state space grows large enough for the GPU-assisted algorithm to consistently outperform the CPU-only version, with the potential savings growing significantly at higher values of N , saving upwards of days or even years, assuming the estimated runtimes are relatively accurate. It can at least be confirmed that the GPU-assisted algorithm saved about 4 minutes at $N = 4$, and at least multiple hours at $N = 5$.

Compared to Sanden’s algorithm, we can see that it manages to outperform our GPU-assisted algorithm for $N \leq 4$. However, we managed to save nearly 3 hours for $N = 5$, and assuming our estimated runtimes are accurate, multiple weeks at $N = 6$, and multiple years at $N = 7$. It should be noted that some of these differences could be due to differences in hardware, as we were not able to confirm the exact hardware that Sanden’s

algorithm was run on. However, it is still a promising sign that we were able to beat these previous runtimes.

Unfortunately, while we may have potentially saved years of computation, a runtime of 6.3 years is still a rather unfeasible amount of time to be running a single algorithm, and thus our algorithm does not yet bring the necessary improvements to solve the next step of the stepping stone puzzle at $N = 7$.

6. ALTERNATIVE APPROACHES

It’s worth covering some notable attempts at optimizing our algorithm which did not yield useful results.

6.1 Grid Arrays

One, perhaps obvious, thing to look at is our selection for the board data structure. It is essentially a matrix which we store in COO (coordinate) format. This is very efficient from a memory perspective, but not always best for the time complexity of some operations. One such operation is computing the sum of adjacent elements given a position on the board. This is something which is computed frequently, and in theory if we stored the whole grid instead of, or in addition to, element coordinates we could take this from linear time complexity in relation to the number of elements on the board to constant time. The extra memory overhead is somewhere on the order of 10 times, which forces the boards off of the registers and into shared memory. Additionally the constant time complexity in this case is eight look up operations, whereas for looking at each element is between one, and even in the worst case for $N=7$, somewhere around 70. Thus we found that in practice it was actually slower to store the entire matrix as a 2D array even when searching a small subset of the $N=7$ state space.

6.2 Boards in shared memory

Instead of storing each board in its thread’s registers, the data could instead be stored in shared memory to reduce register pressure. Consulting the CUDA occupancy calculator, we found that this should increase occupancy significantly with a correctly selected block size. In practice it seems that the extra memory access overhead outweighed

any increase in occupancy, resulting in about a 5% overall slow down.

6.3 Branch Pruning by Upper Bounds

Following the logic presented by Jonathan F. Waldmann [4], which shows that $a(N) < 86 \times N + 32$ we attempted to generate tighter upper bounds for particular board states. In brief, in order to place an element K , at least one of the adjacent cells to K must contain an element less than half the value of K . In this way all elements are tethered some fixed distance away from a one element and eventually there are more numbers than you could possibly have space for. While a bound of $86N$ is of no use to us, once some or indeed many of the one elements are actually placed on the board, this bound tightens significantly. Unfortunately in practice that much tighter bound was never any better than $\sim 8 \times N + 32$, which results in no pruning.

6.4 Branch Pruning by Location Limiting

Another pruning method tried relies on the fact that in order to place K when you have already placed 2 through $K/2$, the board must contain somewhere either a sum of exactly K or a sum $\leq K/2$. Often looking at one element isn't sufficient to show it can't be placed, but if multiple elements are considered simultaneously, some boards can be effectively pigeonholed. In this way upper bounds of $2 \times$ the current board height can be found. However, once again in practice this is not particularly effective. We found that this can potentially prune around 0.16% of board states, which is not enough to outweigh the extra overhead from performing the pruning.

7. FUTURE WORK

There are a variety of additional optimizations that should be considered in future iterations of stepping stone puzzle algorithms. For one, many of the failed efforts mentioned above could be reattempted. While none of the attempted pruning methods proved worthwhile for our implementation, alternative methods with more careful handling of the geometry could yield useful bounds, reducing the search space enough to outweigh the additional overhead of pruning.

Additionally, the algorithm is not currently designed with sharding in mind. If the algorithm is

redesigned such that the search space can be split up to run on separate GPUs, such as on a GPU cluster, it is likely that much higher computation speeds could be achieved. As the threads searching one board require little to no interaction with threads searching other boards, beyond comparing the maximum numbers, which could easily be achieved using local maximums on each GPU and comparing afterwards, this would likely not be complicated to implement. Sharding would require some additional amount of overhead and cleanup at the end, but would likely make significant savings on the overall computation for high values of N .

Looking at the functioning of the kernel itself, it should be noted that there is currently a lot of warp divergence, as adjacent threads often follow divergent paths due to each of them searching boards with different state spaces. Fixing this would require significant restructuring to the current algorithm, but it is still worth considering as a potential optimization. It may be possible to restructure the kernel such that each warp operates on only one board at a time, rather than assigning a separate board for each thread. In this context, a breadth-first search approach may be preferable to depth-first search, as it would allow the threads within a warp to remain synchronized by searching the same depth simultaneously.

Implementations of some of the above optimizations may be enough to feasibly compute the solution for $N = 7$, perhaps even $N = 8$. However, at $N \geq 9$ additional complications arise, as it is now necessary to account for triple multiboards. $N \geq 14$ would require accounting for four multiboards, and the potential number of multiboards continues to increase at higher levels of N . While these complications seem far out of reach, they are still worthwhile considerations for the future. Work should go into the implementation of some way to track 3 or more multiboards. Alternatively, this could also be solved by formulating a proof that multiboards are never the optimal strategy for generating the maximum number from any given N , which would allow us to completely ignore the potential of multiboards.

8. CONCLUSION

We have shown that parallel GPU computation techniques are effective at improving the

performance of solving the stepping stone puzzle, with our GPU-assisted algorithm showing significantly lower computation times at higher levels of input compared to the previous best algorithm. However, there is room for improvement, as the algorithm is still expected to take multiple years to compute $N = 7$, the next unsolved step of the puzzle. Theoretically, this algorithm could be run for that period of time to compute the solution for $N = 7$, but it would likely be more worthwhile to first improve the algorithm to reduce computation time further. We have outlined multiple potential optimizations that could be made to the algorithm, with varying degrees of difficulty of implementation. These, and any additional optimizations not covered here, should be implemented and tested in the pursuit of computing the solutions for $N \geq 7$.

9. REFERENCES

- [1] Hugo van der Sanden. 2020. A selection of programs that relate to potential or actual sequences in the OEIS. GitHub repository. <https://github.com/hvds/seq/tree/master>. Accessed Dec 17, 2025.
- [2] Hugo van der Sanden. 2020. Extend the most recent “nice” OEIS sequence: stepping stone puzzle on a grid. Stack Exchange. <https://codegolf.stackexchange.com/questions/212160/extend-the-most-recent-nice-oeis-sequence-stepping-stone-puzzle-on-a-grid>. Accessed December 17, 2025.
- [3] N. J. A. Sloane et al. 2025. The On-Line Encyclopedia of Integer Sequences, Sequence A337663. <https://oeis.org/A337663>. Accessed December 17, 2025.
- [4] Jonathan F. Waldmann. 2022. Proof that $a(n) < 86 \cdot n + 32$. OEIS auxiliary document for Sequence A337663. https://oeis.org/A337663/a337663_3.txt. Accessed December 17, 2025.
- [5] Peter Weiblen, Adam Meyers. 2025. Program To Optimize Stepping Stones. GitHub repository. <https://github.com/piweiblen/PTOSS>. Accessed Dec 17, 2025.